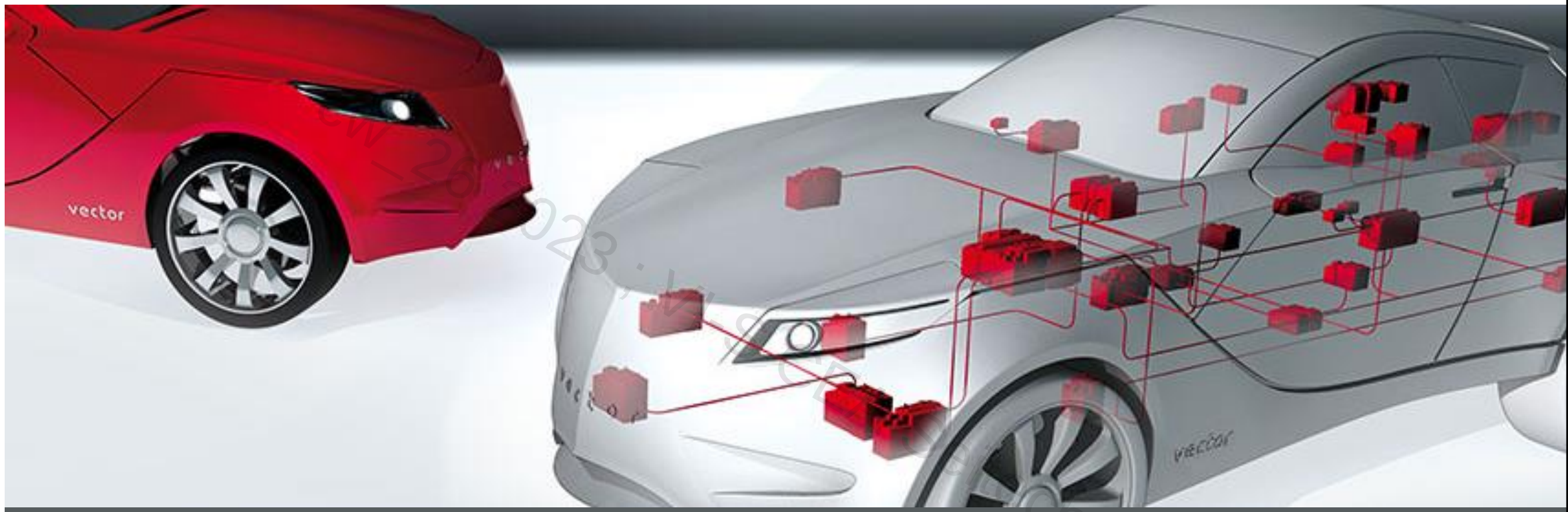
VECTOR



AUTOSAR in Practice

Software Components

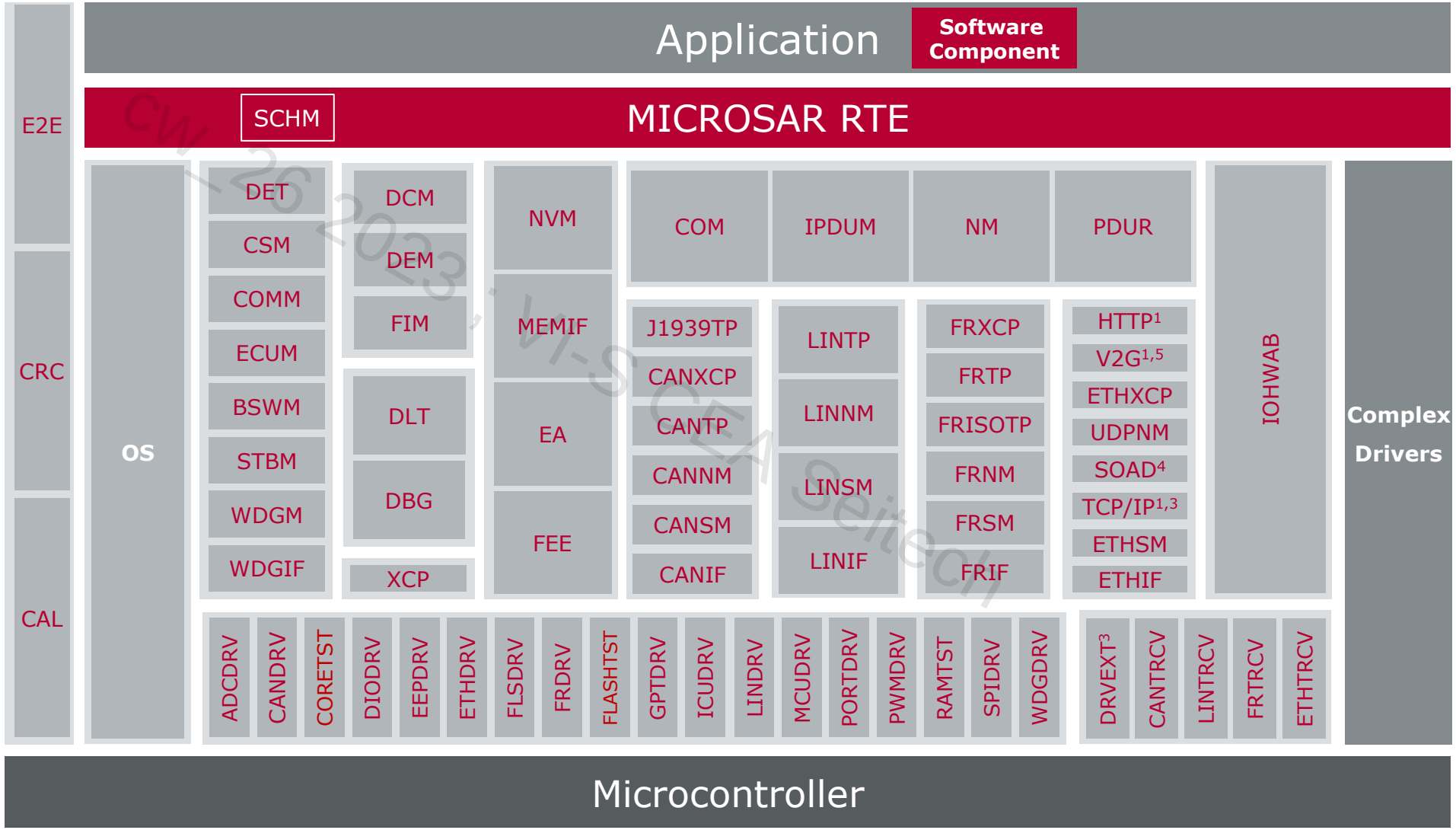
Agenda

- **Software Components**
- Packages
- Data Types
- Coming up next

cw_26 2023 ; VI-S CEA Seitech

Elements for Software Design

- Software Components (SWCs) require an RTE
- RTE runs the SWCs
- RTE runs the BSW modules over SCHM



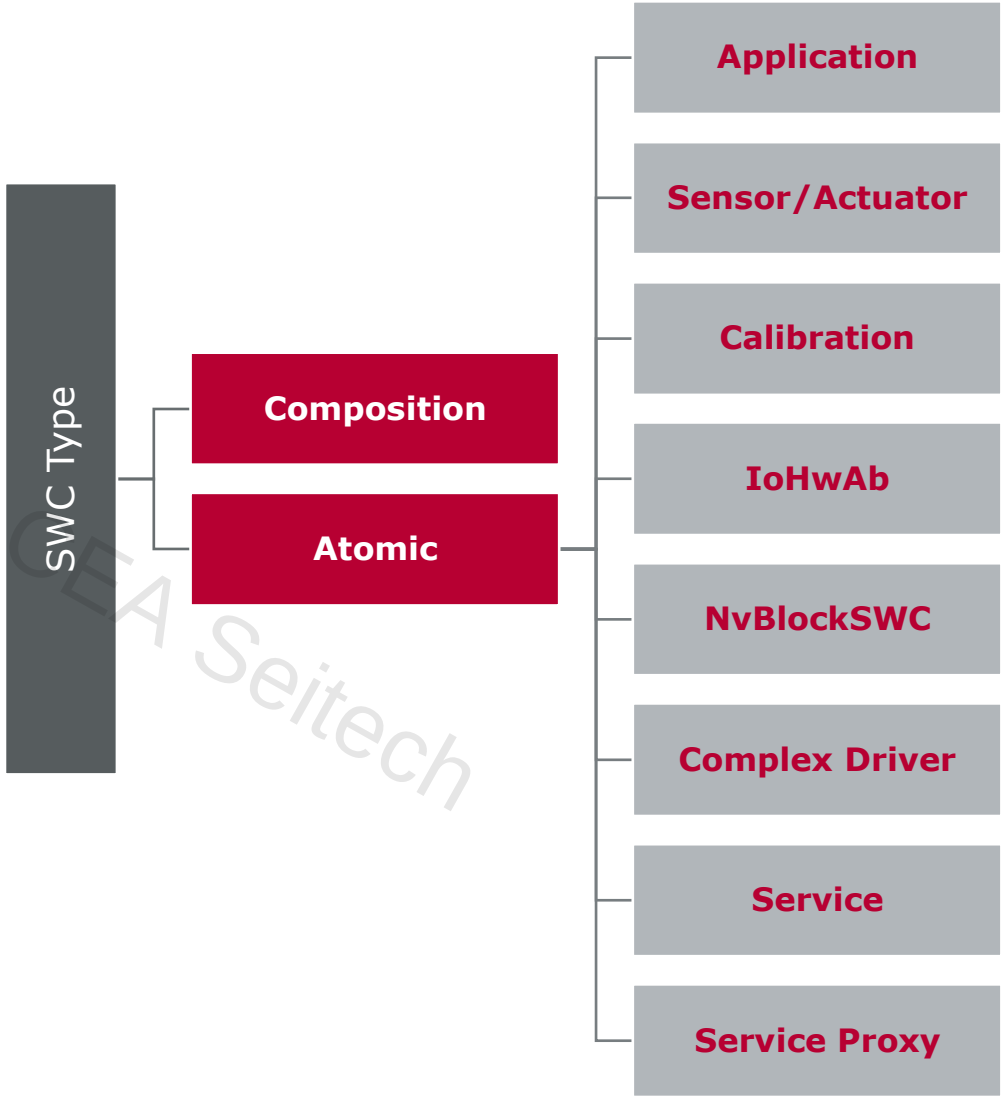
CW 26 2023 - VI-SCEA Seitech

Overview on the kinds of SWCs

Software Component Types (SWC)

- Composition
  - has Port Prototypes
  - Does not have any internal behavior
  - Does not result in executable code
  - Is placeholder, container for other SWCs
- Atomic
  - Has Port Prototypes
  - Internal behavior exists (Runnables, …)
  - Results in executable code
  - Cannot be divided any further

- In our tools, we distinguish between
  - Application Component Types
    - Constituting Parts of the application
    - information exchange over AUTOSAR Interface
  - Service Component Types
    - Representation of the service layer modules
    - Services of the BSW be accessed by the application (Standardized AUTOSAR Interface)



cw 26 2023 ; VI-S CEA Seitech

Overview on the kinds of SWCs

Compositions can be considered only the hull without an own internal behavior, but with a defined interface.

They serve as placeholders for further software components which could in turn be also compositions but in the end, there must be atomic software components.

Atomic software components yield a part of the executable code which exists in the binary. They form the leaves of the hierarchy tree of the software design.

In our tool, we distinguish the following specialized software components:

Application – hardware independent, fully reusable

Sensor/actuator – implementation technically hardware independent, but depending on the existence of a hardware sensor/actuator. Provides characteristics

Calibration – an AUTOSAR term for ROM constant Component which holds and distributes ROM values in the system (calibration parameter)

IoHwAb – IO Hardware abstraction. Resides in the ECU Abstraction layer and abstracts from everything outside the µC on the PCB.

NVBlockSWC – aggregation of NVRAM data prior to writing it down in a NVRAM Block. Distribution of Nv data elements

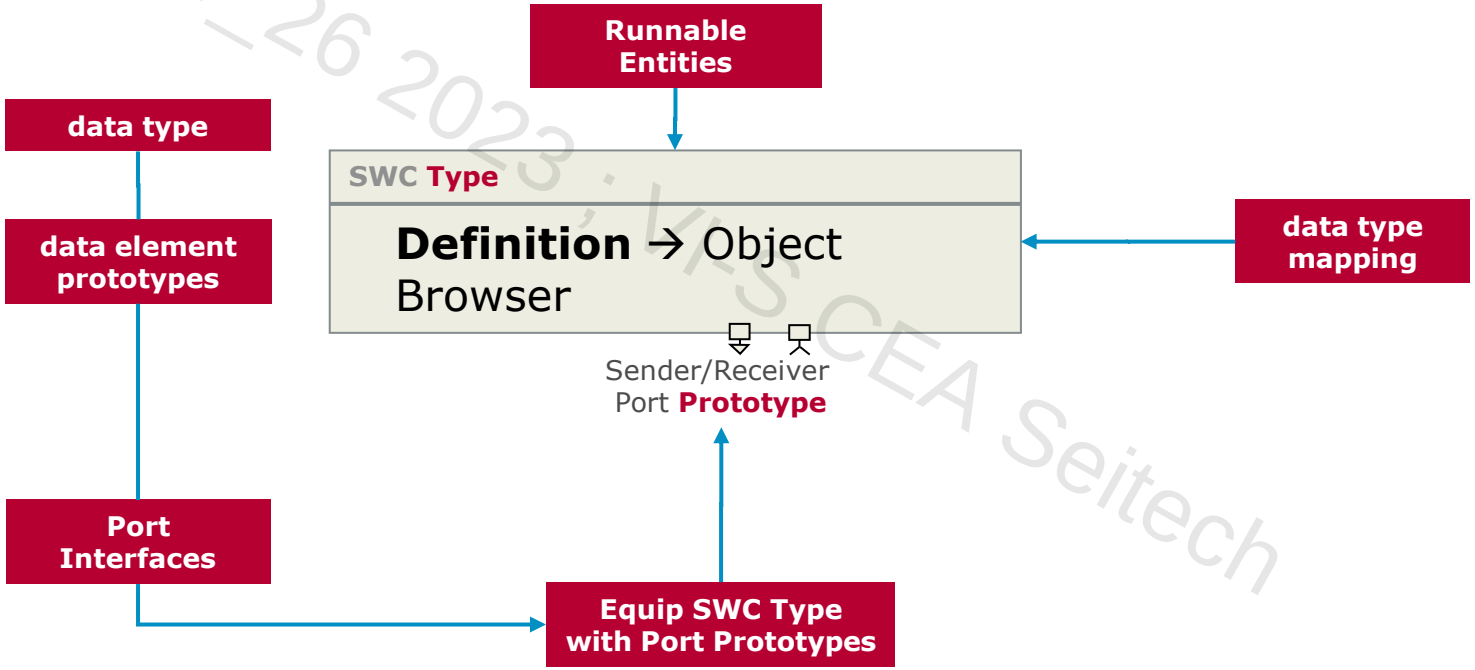
Complex Driver – a SWC which contains hardware dependent parts of implementation, and also direct Standard Interface calls into the BSW.

Service – a service SWC which is the representation of a member of the Service Layer in the BSW. A service module offers Standardized ATUTOSAR Interfaces against which the SWCs can connect.

Service Proxy – a stand-in for a basic software service module for multicore scenarios

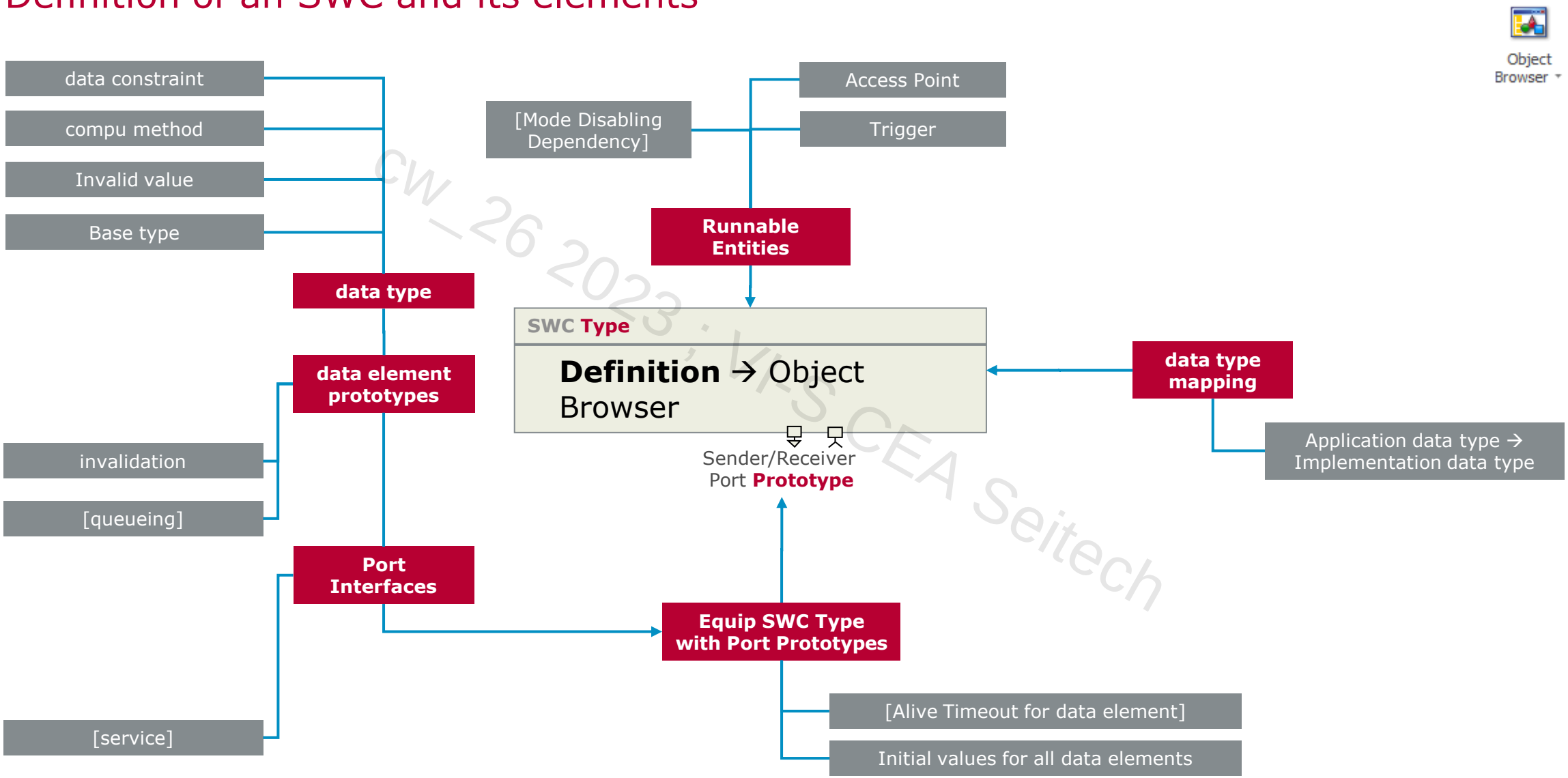
CW_26 2023 : VI-S CEA Seitech

Definition of an SWC and its elements



cw_26 2023 · VRS CEA Seitech

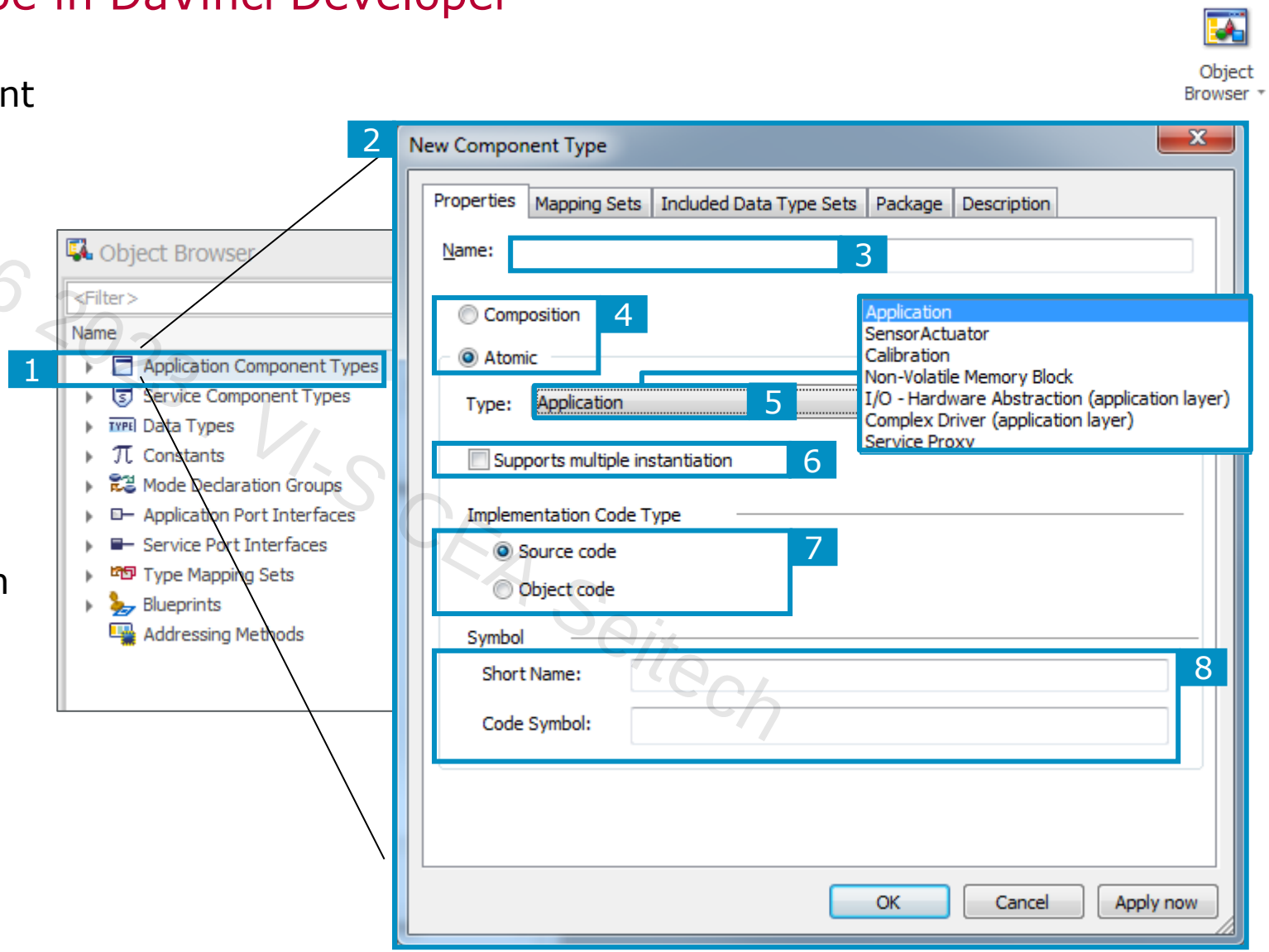
Definition of an SWC and its elements



CW_26 2023 · VRS CEA Seitech

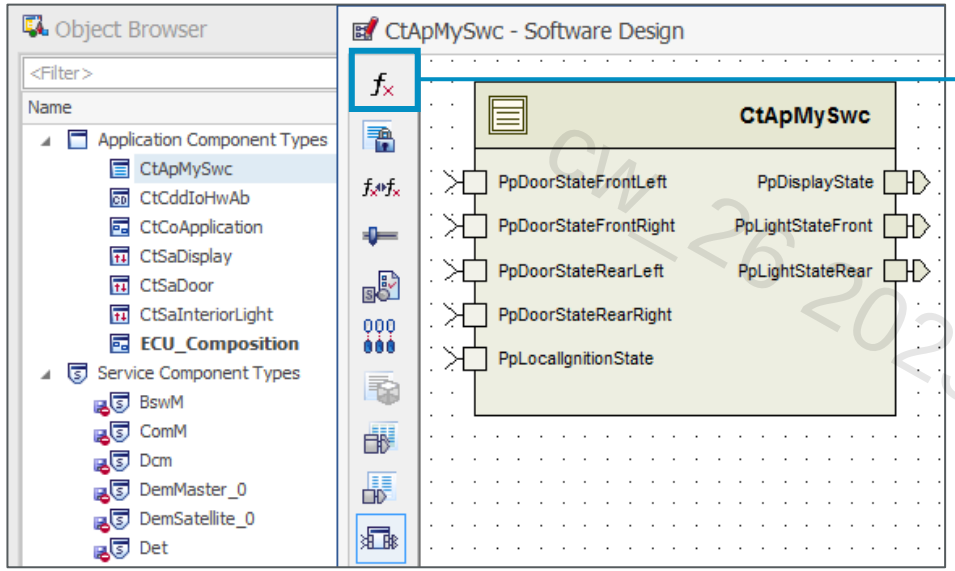
Definition of a new SWC Type in DaVinci Developer

1. Right Click Application Component Types
2. New Component Type comes up
3. Give SWC Type Name
4. Composition or Atomic
5. Type of atomic SWC annotation for intended use of SWC Type
6. Multiple instantiation
7. Source or library implementation
8. Symbol: to be used by the RTE generator instead of the name. Name: must be identical to Value



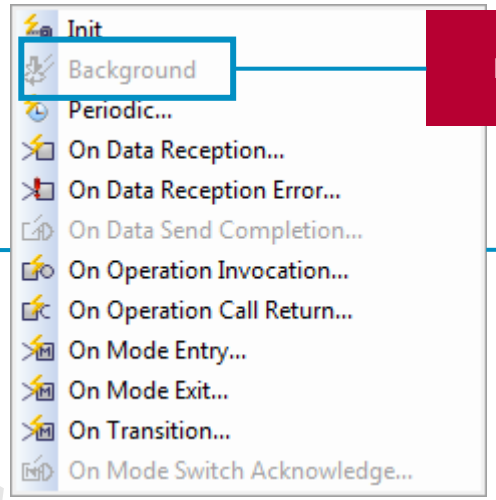
cw_26 2023 VI-SCEA Seitech

Define Runnable Entities inside an SWC Type



Definition of Runnables

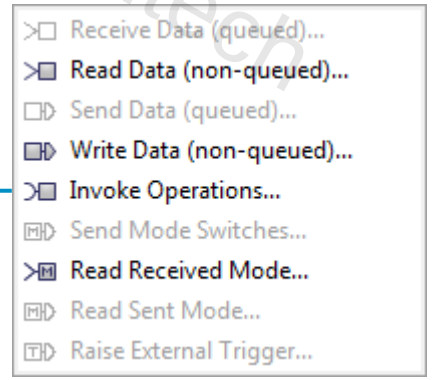
Definition of a trigger for the execution of the Runnable



Background Trigger: Execute Runnable in a background task

[Mode Disabling Dependency]

Define Access Point in case the Runnable has to use a Port Prototype of the SWC Type

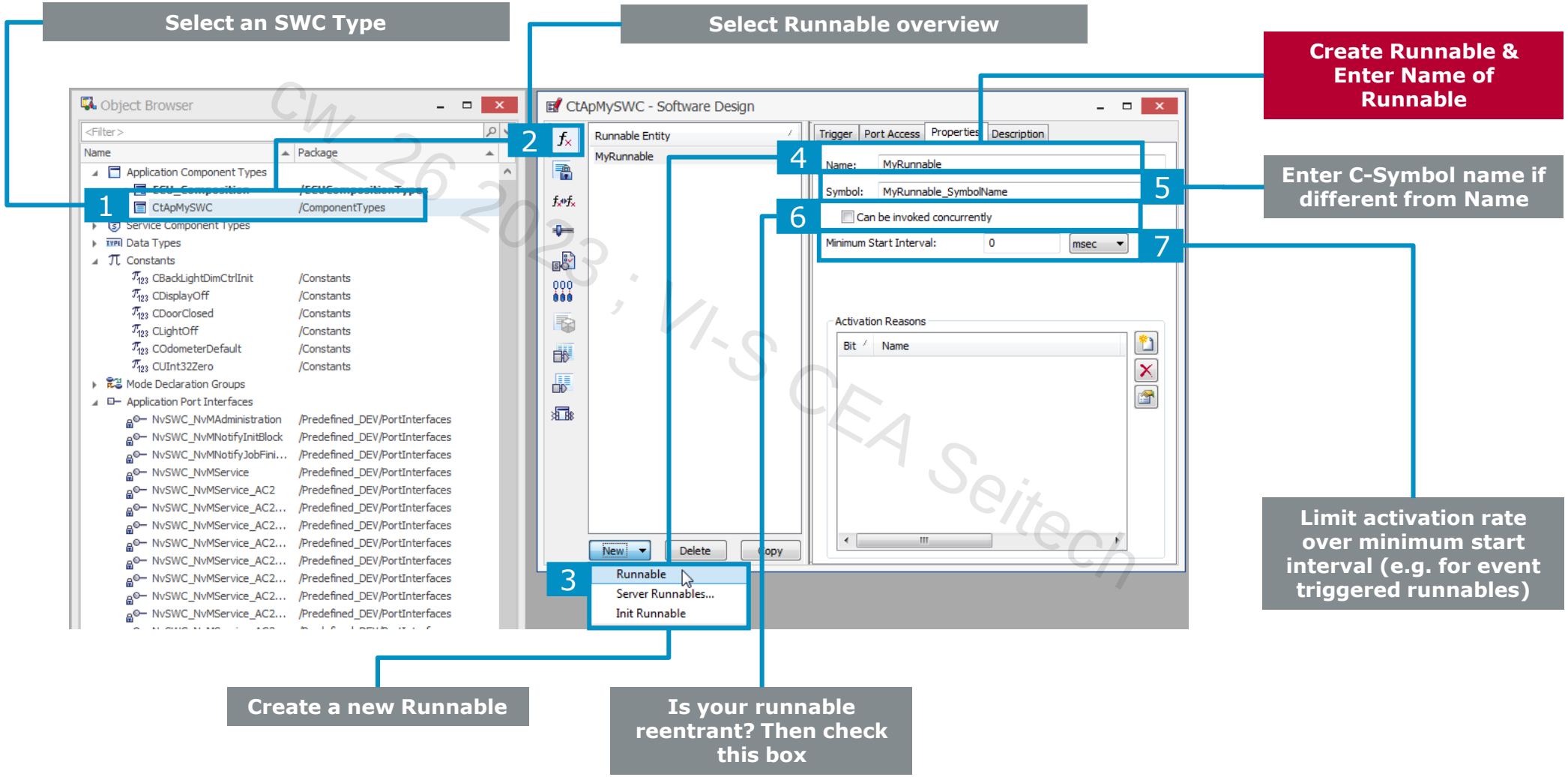


Please take care the Access Point is defined for all Runnables where it is needed (and take care you do not perform the access on C code level without defining it in Developer!)

AUTOSAR 4 defines also “Triggered Events”. This is yet to come in Vector’s software.

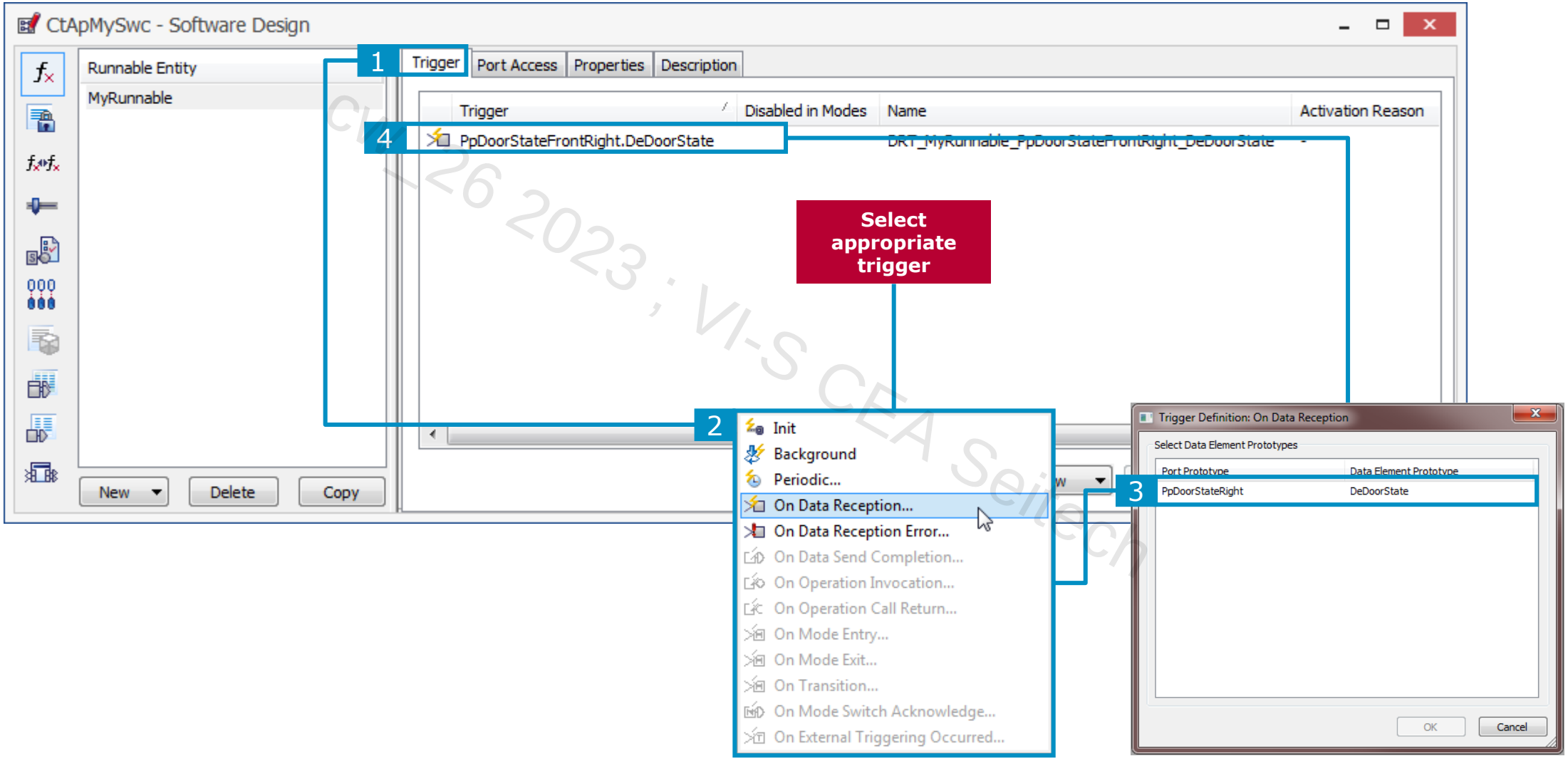
CW 26 2023 ; VI-S CEA Stuttgart

Defining Runnable Entities ("Runnables") in DaVinci Developer



CW_26_2023 ; VI-S CEA Seitech

Defining Trigger setting for a Runnable in DaVinci Developer



CUP 26 2023 ; VI-S CEA Seitech

Defining Trigger setting for a Runnable in DaVinci Developer

Triggers

Runnables need triggers. A trigger can be

- **Periodical** – this is caused by an event which occurs cyclically; here an OS Alarm is configured and a respective Event is set if the Alarm expires.
- **On Data Reception** – means new data information has been received over a communication bus or from another Software Component
- **On Data Reception Error** – an error occurred during the reception of data
- **On Data Send Completion** – the runnable entity is triggered upon completion of a transmission of an output data element (Tx Acknowledge).
- **On Operation Invocation** – this is needed if you implement server functionality in your SWC and a client component calls a service of your SWC. The runnable which has to provide this service is triggered by the RTE.
- **On Operation Call Return** – an asynchronous operation has been started and as soon as this operation is finished, the runnable will be called.
- **On Mode Entry** – Event which occurs when the Mode Manager enters a certain mode (transition)
- **On Mode Exit** – Event which occurs when the Mode Manager leaves a certain mode (transition)
- **On Mode Switch Acknowledge** - the runnable entity is triggered upon acknowledgement of a mode switch of a mode P-port prototype

Depending on the number of triggers and internal communication (e.g. queued S/R) of runnables which are mapped to a certain task, the RTE generator makes a decision to create

1. a **Basic Task** or
2. an **Extended task** with wait points, events, and alarms.

cw_26_2023 : VI-S SEA Seitech

Defining Access Point for a Runnable in DaVinci Developer

CtApMySwc - Software Design

Runnable Entity
- RCtApMySwcCode
- RCtApMySwcComM_ModeChange_FULL...
- RCtApMySwcComM_ModeChange_FULL...
- RCtApMySwcInit
- RCtApMySwcPostRunCode

New | Delete | Copy

1 Access Points | Properties | Description

DEP/Operation/Trigger	Access	Name
PpDisplayState.DeDisplayBusState	Write (explicit)	SEND_PpDisplayState_DeDisplayBusState

4

Select appropriate access point

2 New | Edit... | Delet

- Receive Data (queued)...
- Read Data (non-queued)...
- Send Data (queued)...
- Write Data (non-queued)...
- Invoke Operations...
- Send Mode Switches...
- Read Received Mode...
- Read Sent Mode...
- Raise External Trigger...
- Internal Triggering Point...

Port Access Definition: Write Data

Sender Port Prototype	Data Element Prototype	Tx Ack
PpBswMPostRunRequestCtApMySwc	requestedMode	-
PpBswMRunRequestCtApMySwc	requestedMode	-
PpDisplayState	DeDisplayBusState	-
PpDisplayState	DeLightOnOffCounter	-
PpDisplayState	DeOdometerValue	-
PpDisplayState	DeOdometerWriteRequestPending	-
PpLightStateFront	DeLightState	-
PpLightStateRear	DeLightState	-

3

Properties

Port Access: Write (explicit)

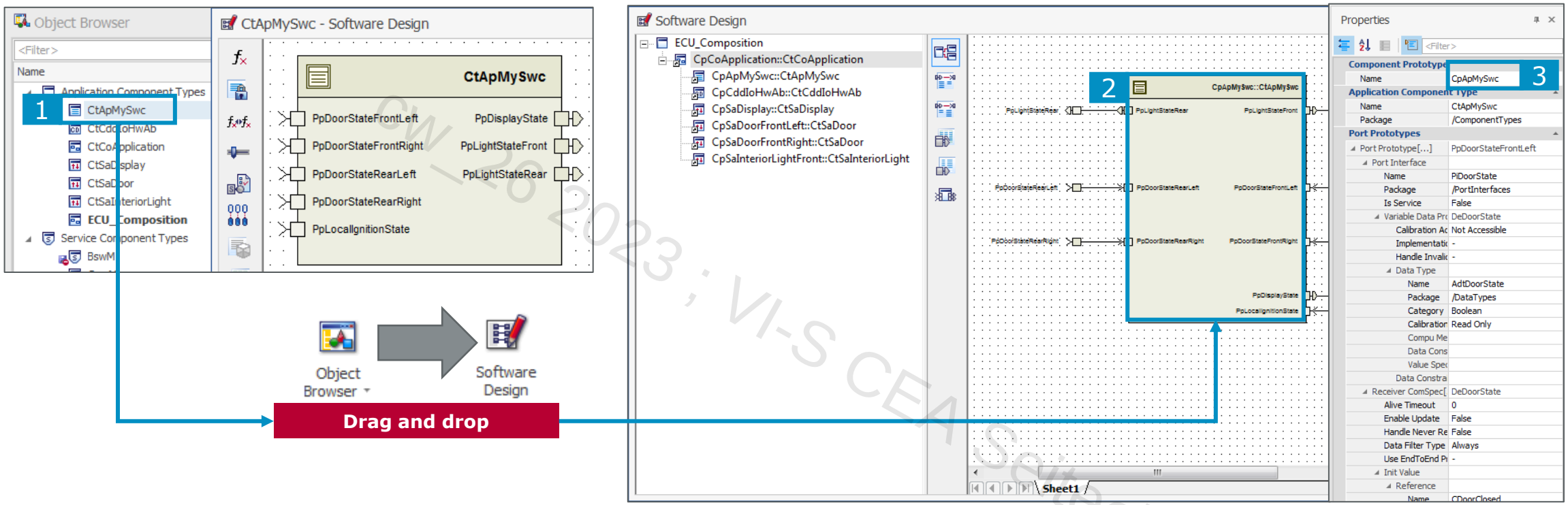
Tx Acknowledgement Wait Point

OK | Cancel

Please take care the Access point is defined for all Runnables where it is needed (and take care you do not perform the access on C code level without defining it in Developer!)

ev_26 2023 V1-S.Schneitech

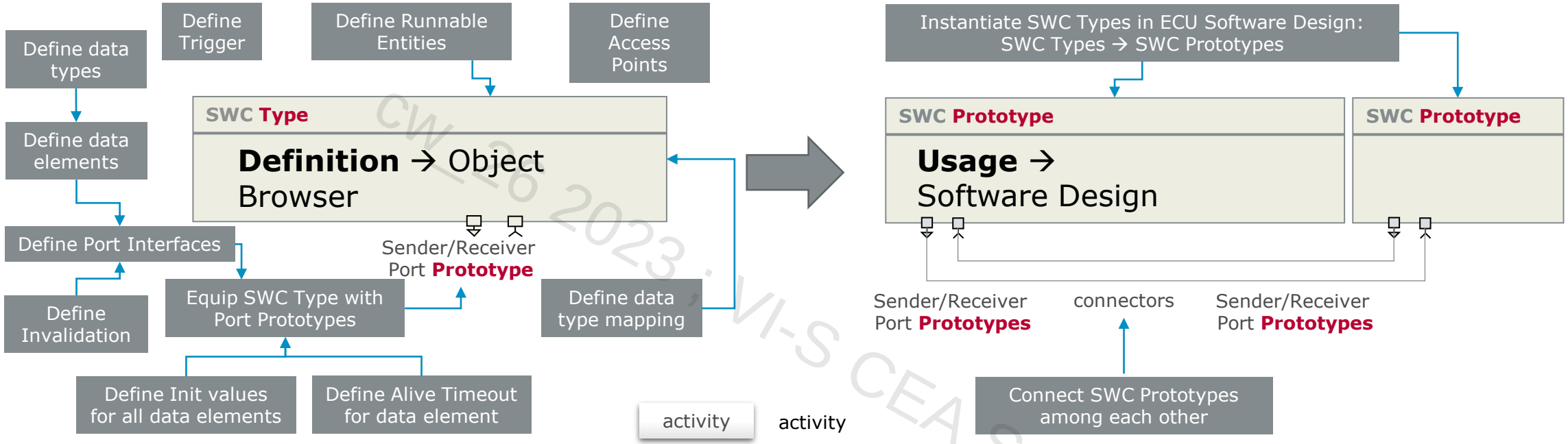
Usage of an SWC Type in the Software Design



1. Dragging and dropping the SWC Type from the Object Browser into the Software Design …
2. … instantiates the SWC Type as SWC Prototype
3. You can edit the properties like the prototype name by selecting the SWC. The properties view will update accordingly

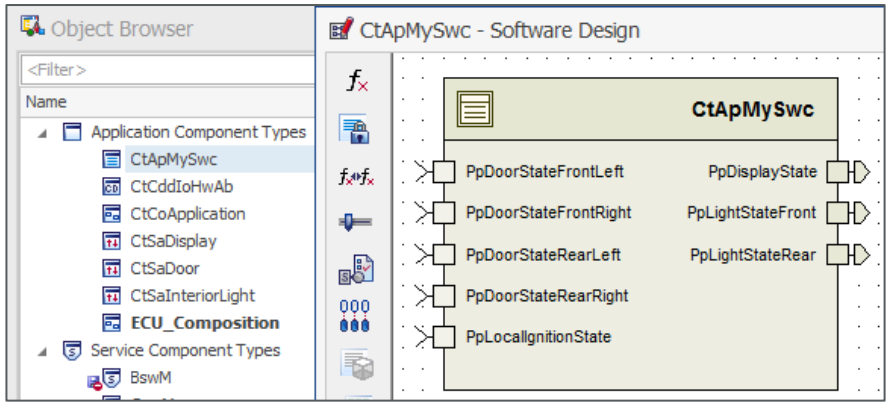
CW_26_2023 ; VI-S CEA Seitech

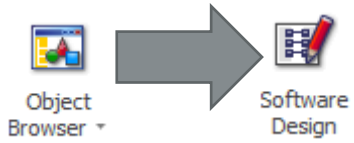
Definition and usage of SWCs and their elements

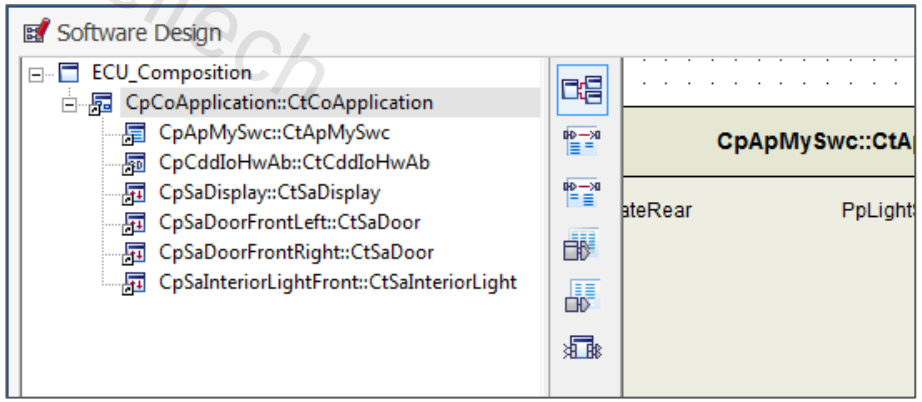


activity — activity

→ needed by





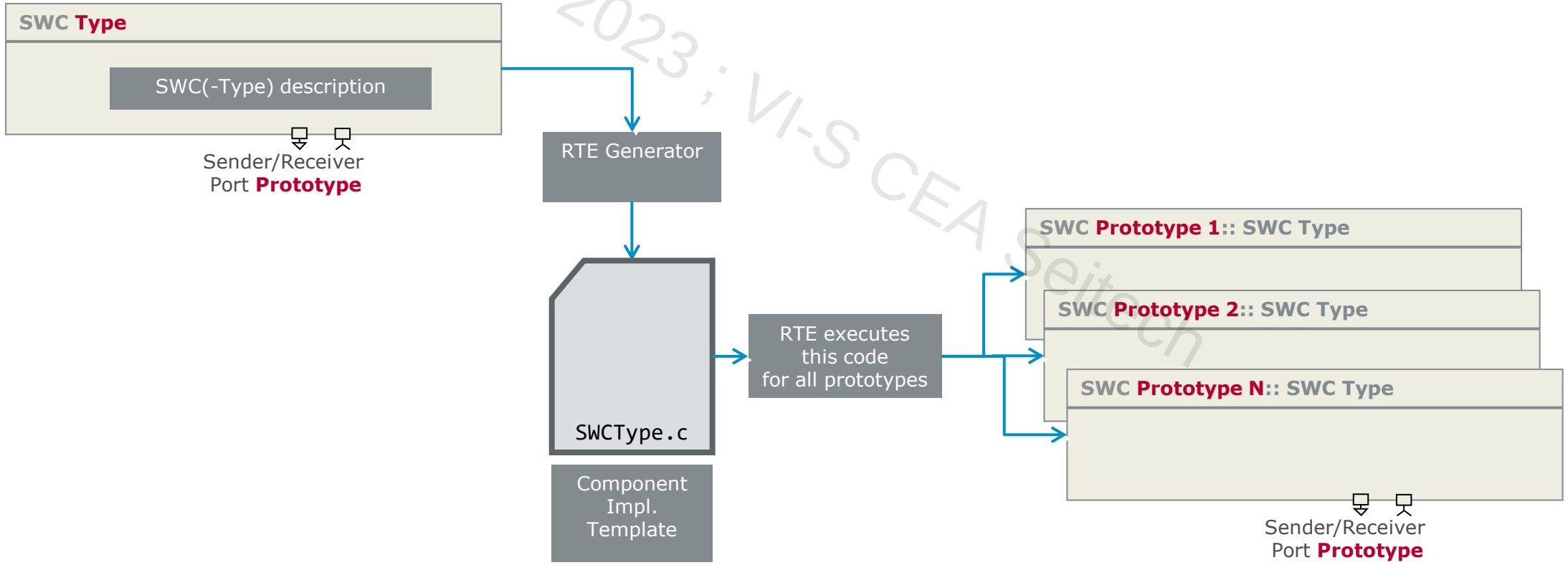


CW_26 2023 VI-S CEA Seitech

Generation of the implementation template

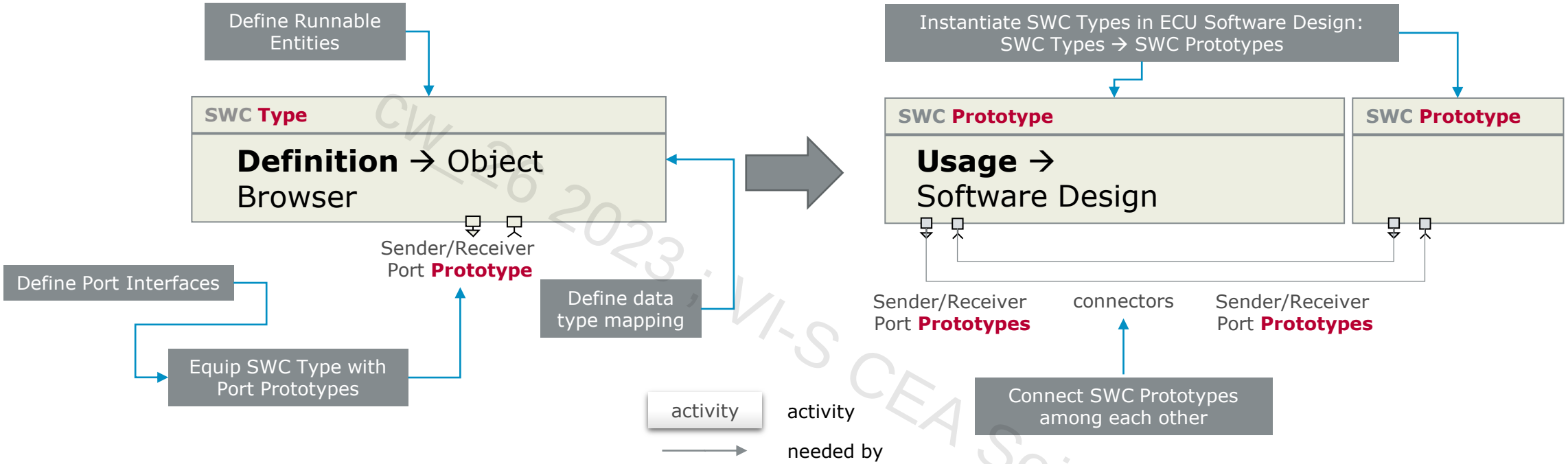
The implementation (generated SWC Template) is applicable to the Component **Type**

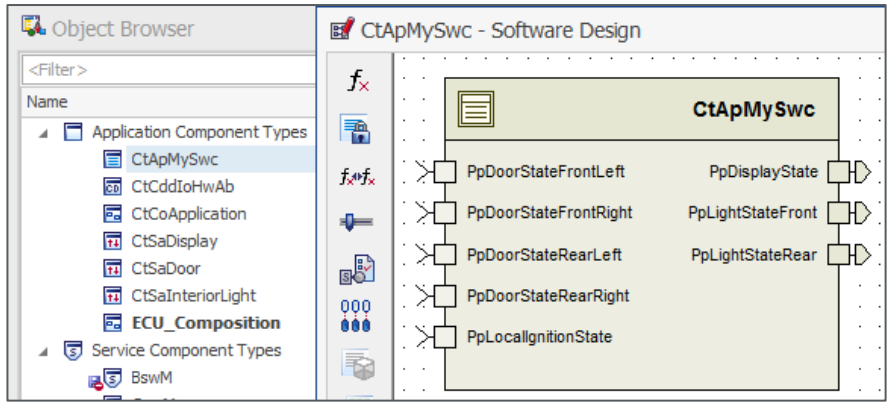
- Implementation is unique for one Component Type and not for the instantiated SWCs (SWC Prototypes)
- Component implementation template can be generated

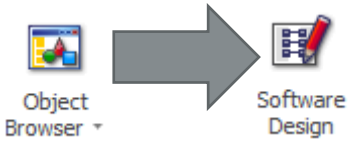


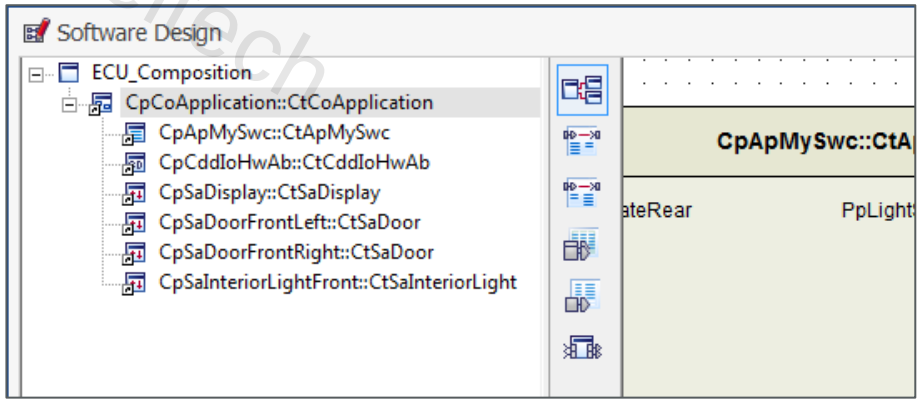
cw_26 2023 ; VI-S CEA Seitech

Designing and using Software Components





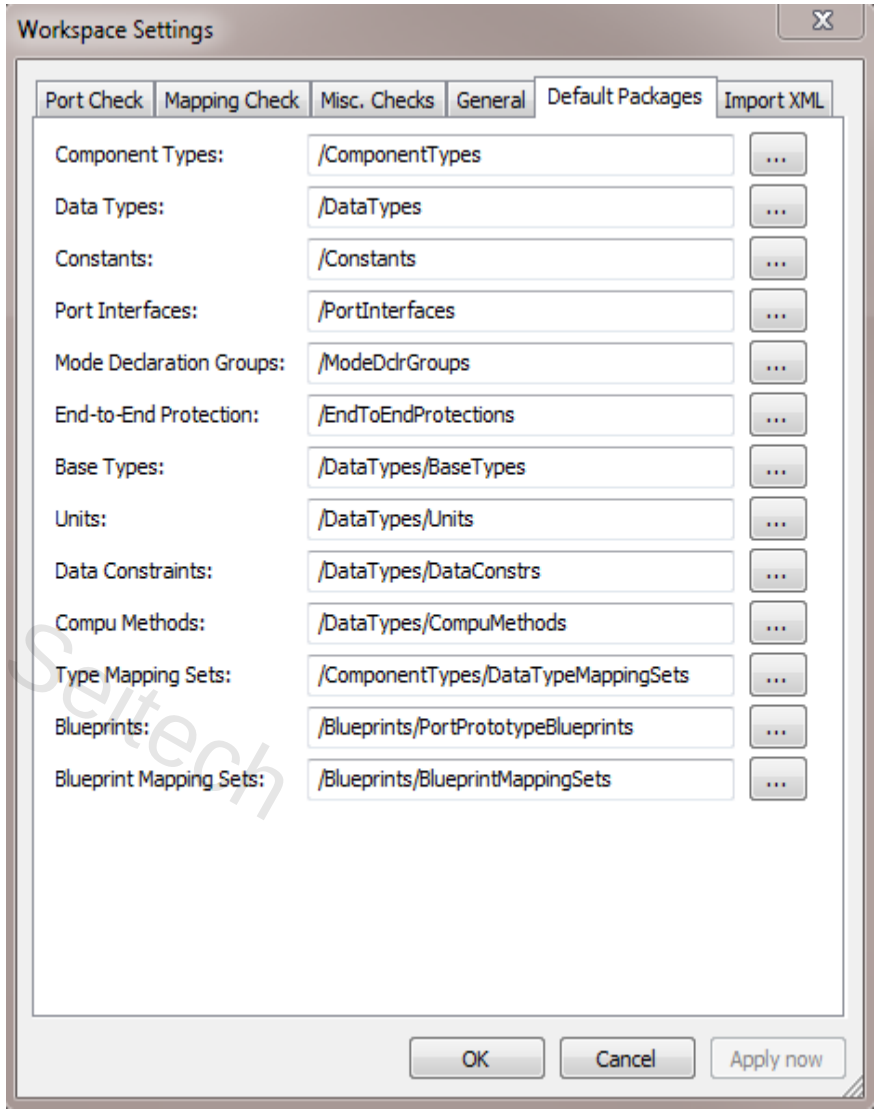




CW_26 2023 VI-S CEA Seitech

Packages in DaVinci Developer

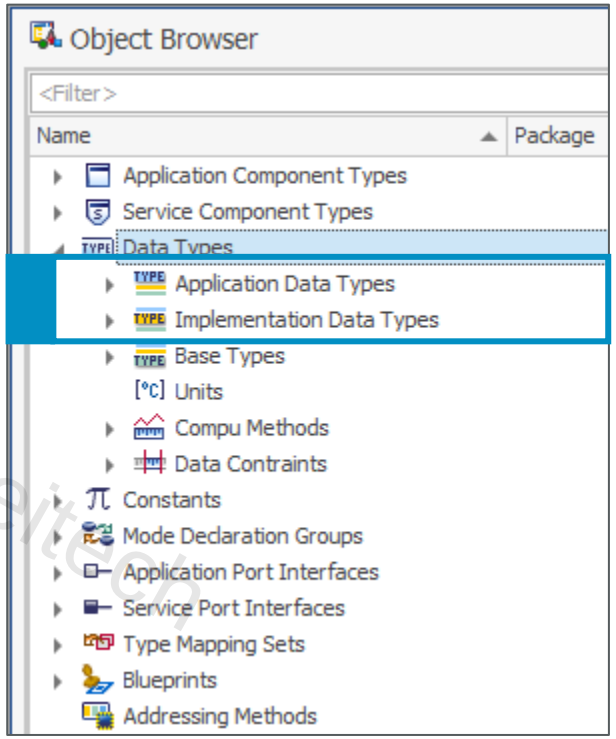
- Default Packages Page
- Select the default package for new objects for all types of library objects
- This is used when
  - creating new objects in the Types view of the Object Browser
  - creating objects automatically, e.g., during data mapping via the “Create Port Prototypes” function
  - selecting the option “Use Default Packages” during import of an .arxml file



cw_26_2023 , VI-S CEA Saitech

Data Types

- AUTOSAR distinguishes between Application Data Types (ADT) and Implementation Data Types (IDT)
- Application Data Types (ADTs) exist on conceptual level
  - Are abstract data types
  - Conceptual/algorithmic view
  - For function developers
  - Optional usage
- Implementation Data Types (IDTs) exist on code level
  - concrete data type
  - For implementers
  - Mandatory for code generation

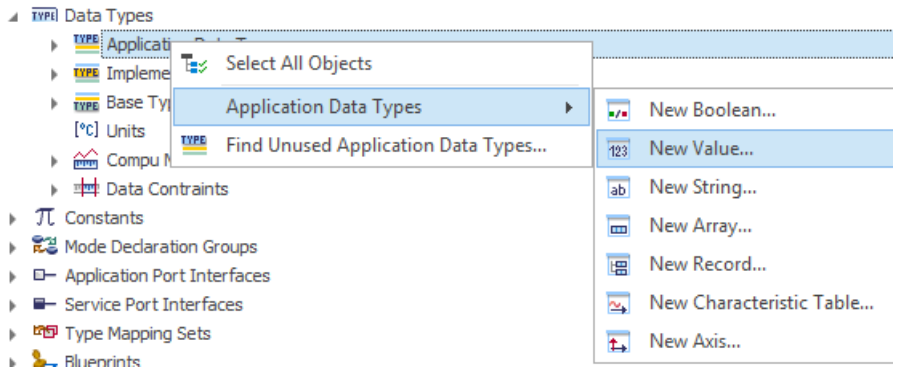


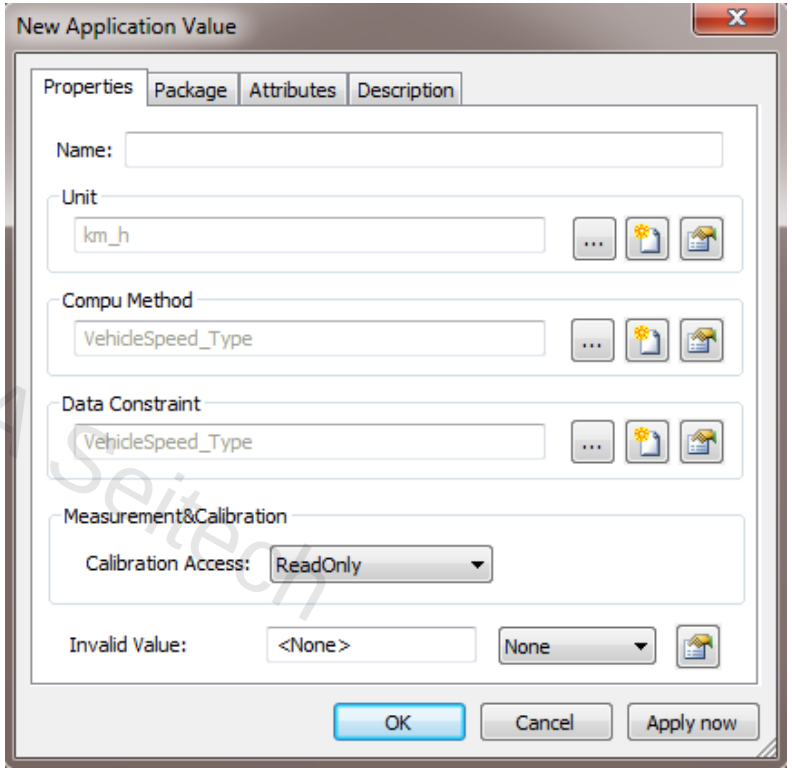
CW_26 2023 ; VI-S CEA Seitzer

Application Data Types

- abstract data type, conceptual/algorithmic view
- Can be used in Port Interfaces
- for RTE generation mapping onto Implementation Data Type (IDT) required

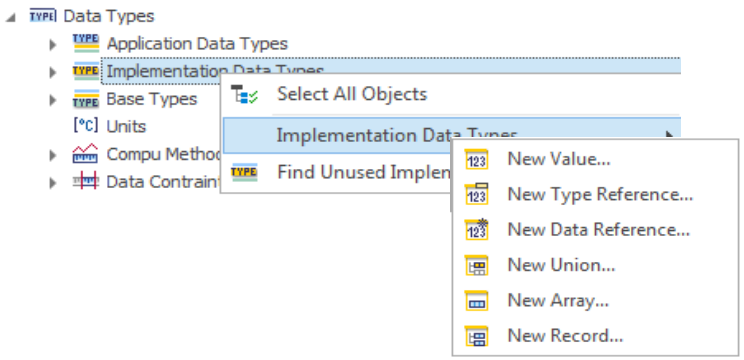
CVX26 2023 ; VI-S CEA Seitech

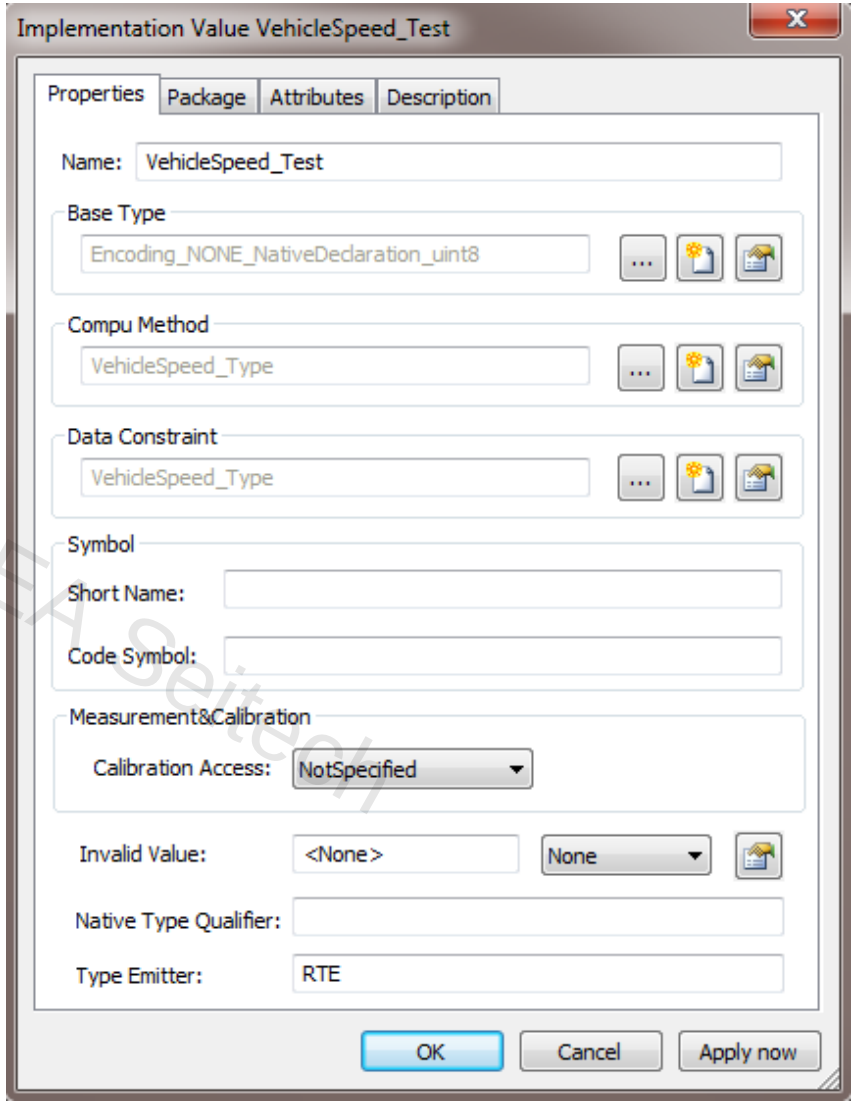




Implementation Data Types

- include semantics, implementation view
- Can be used in Port Interfaces
- IDT references Base Type (if you choose IDT name identical to name of platform type then reference can be omitted)



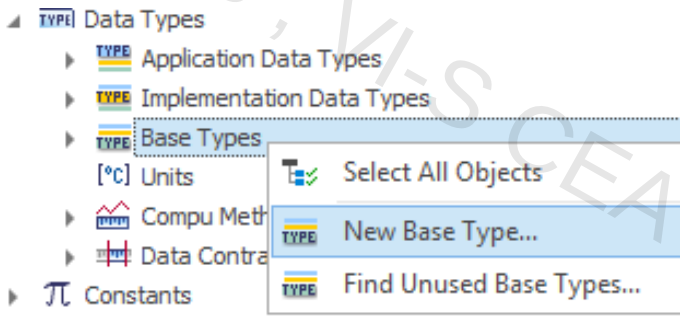


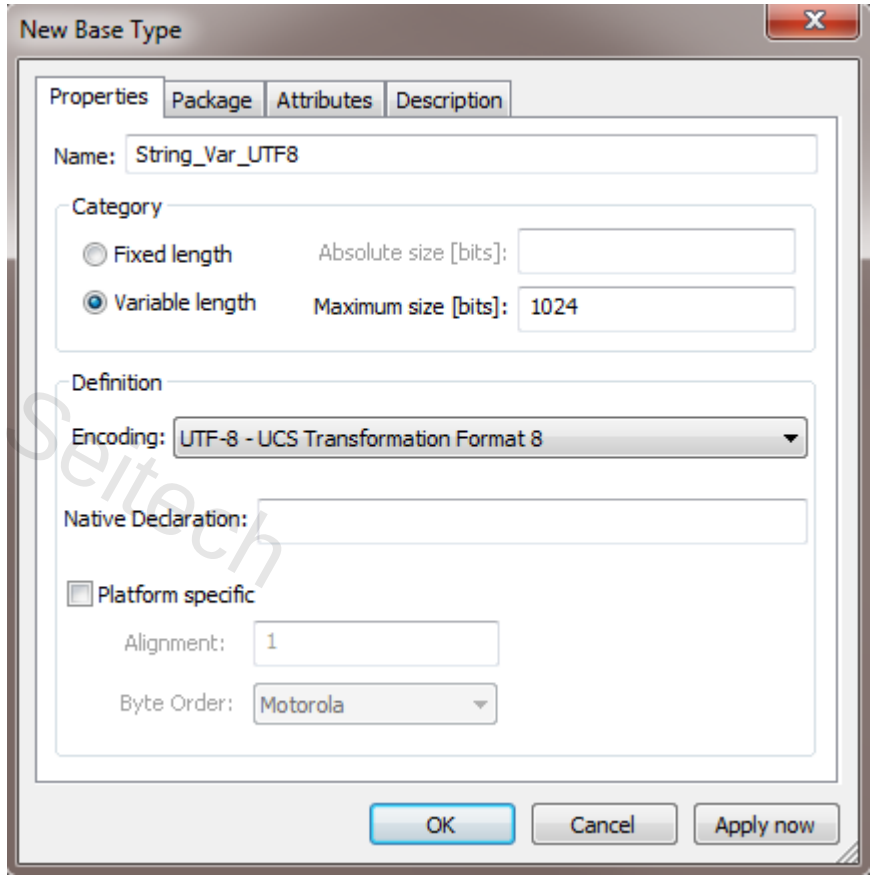
- Could be a complex data type which consists of base types / implementation types
- OR directly references a Base Type / platform type

Oct 26 2023 ; VI-S CEA Seitech

Base Types

- Base Type
  - code-level type
  - no semantics
- Allows platform-specific configuration
  - Not required if implementation type is a known platform type
- Native Declaration
  - can be used to influence the RTE Type definition and correct usage user

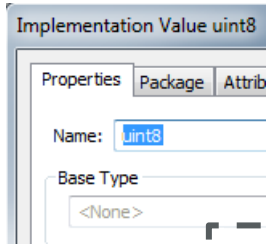




- Platform type corresponds to a Base Type specified for my platform
  - For example, if only 16-bit minimum access is possible, then we have an uint8 platform type which is implemented as uint16
- Platform Types abstract the platform specifics
- Base Types are the well-known C language types

cw_26_2023 ; VI-S CEA Seitech

Platform Types create platform independent software



1) Reference by name (=AUTOSAR Std. Type)

ImplementationDataType

Name:
uint8

Category:
Value

use

Platform_Types.h

```
typedef unsigned char    Boolean;
typedef unsigned char    uint8;
typedef unsigned long    uint32;
[…]
```

Platform Type

Name:
uint8
int. data constr.:
0..255

use

ImplementationDataType

Name:
MyDataType

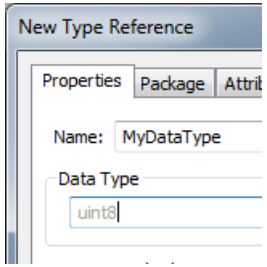
Category:
Type Reference

generate

Rte_Type.h

```
typedef uint8 MyDataType;
```

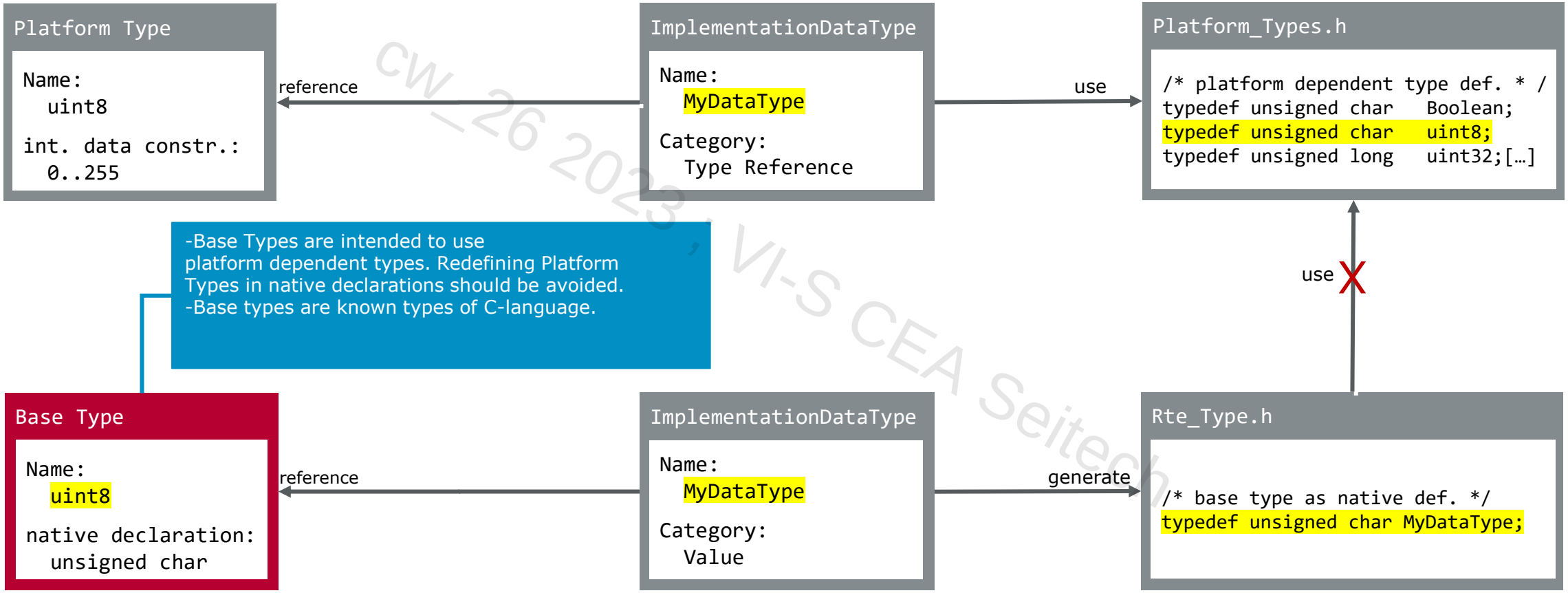
2) Type reference (name !=AUTOSAR Std. Type)



HINT: Always use Platform Types. Either direct use of or reference to a platform type

CW_26 2023 ; VI-S CEA Seitech

Base Types bypass the Platform Type concept

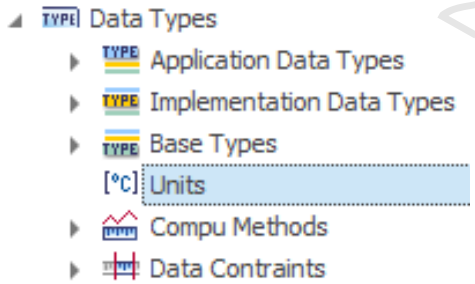


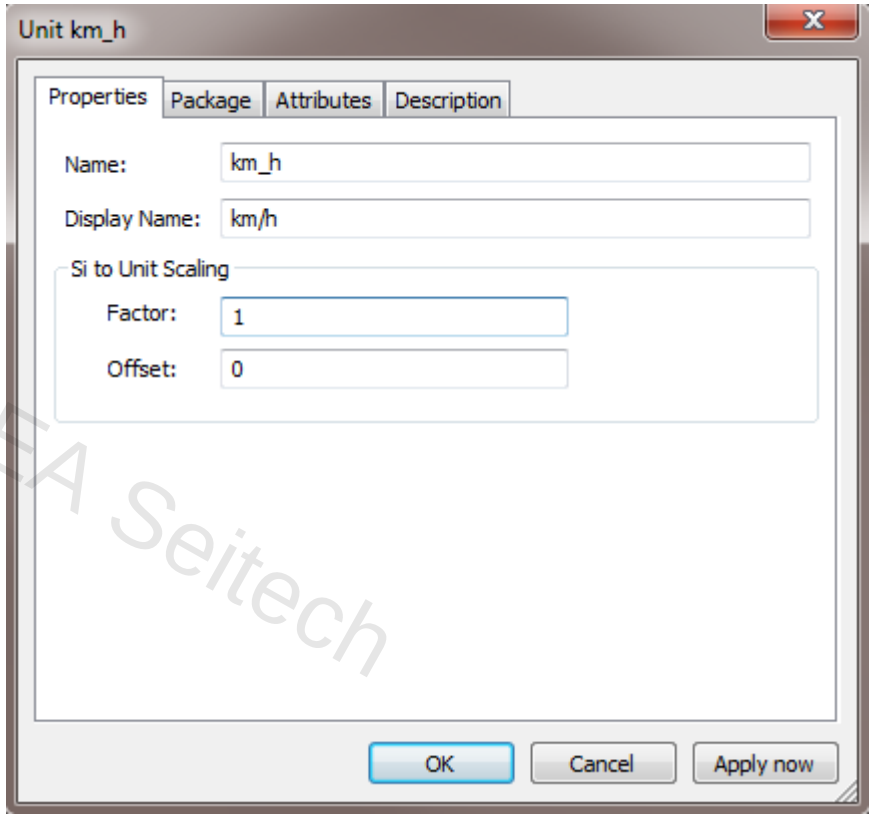
cw_26 2023, VI-S CEA Seitech

Units

- Units are globally defined
- Both Application Data Types and Compu Methods reference units

cw_26 2023 ; VI-S CEA Seitech



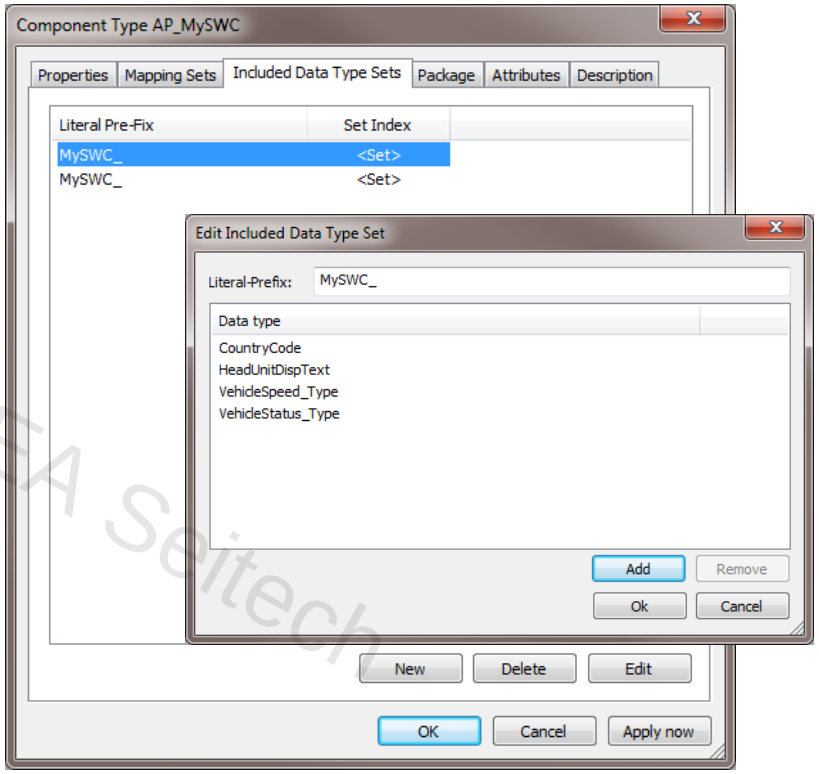


Only defined with no real influence on the code

- can be assigned to an Application Data type
- Defines how the Application type shall be interpreted
- Computation methods as well as units define how a signal conversion can be done from logic interpretation to physical unit

Included Data Type Sets

- Specifies the data types used within the implementation of an SWC (i.e., not used by a port)
- N lists of data types can be defined, each with a prefix



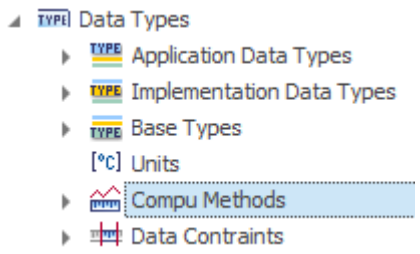
Grouping together which datatypes are used within the SWC

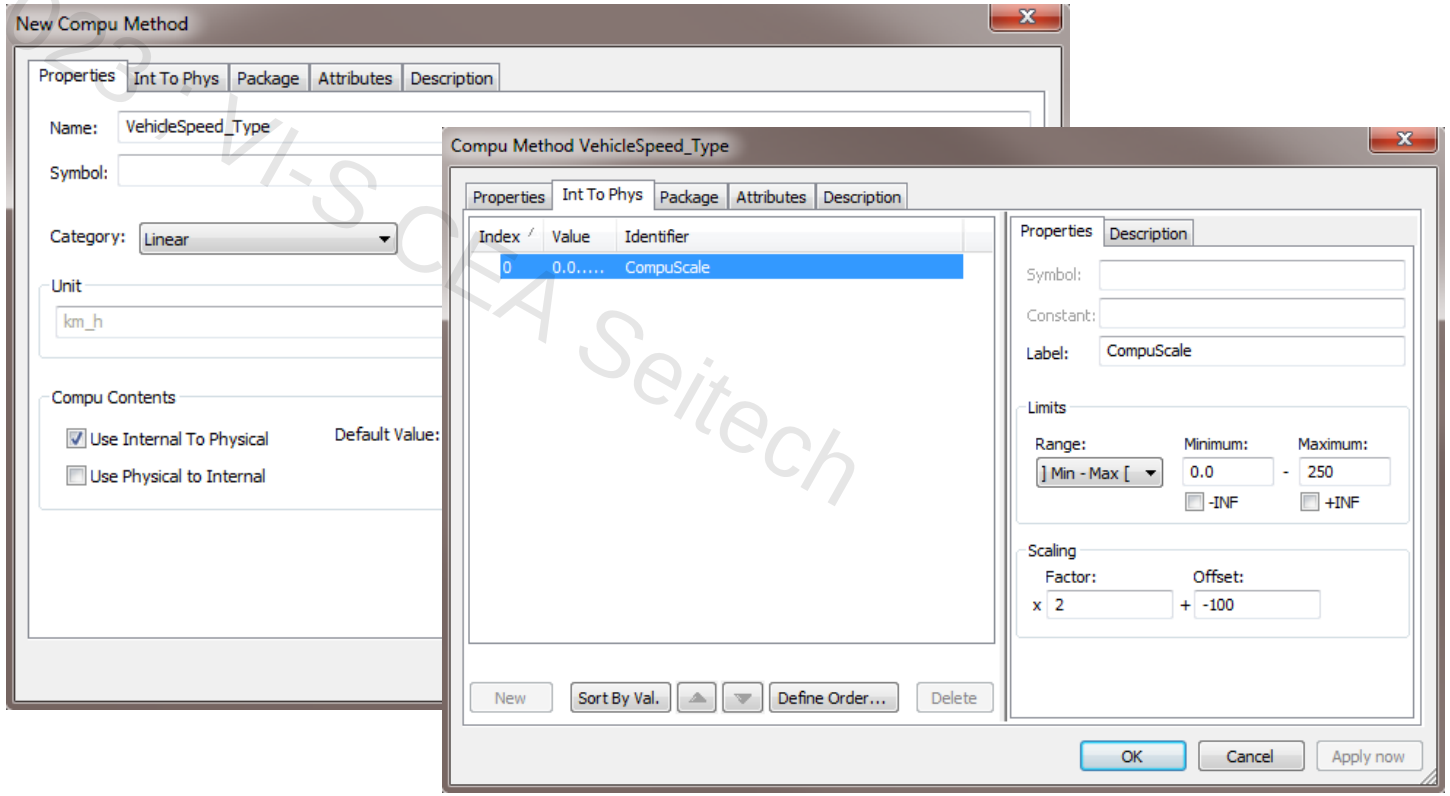
- Datatypes can be equipped with a prefix so they will be specific to the SWC
- Types will be generated / defined by the RTE in the component header file

cw_26 2023 ; VI-S CEA Seitech

Compu Methods

- Compu Methods are globally defined
- Categories: Identical, Linear, Scale-Linear, TextTable, Scale-LinearAndTextTable, BitfieldTextTable
- IDTs typically use **Internal-To-Physical** definitions
- ADTs typically use **Physical-To-Internal** definitions





Defines how the conversion from a physical to internal signal interpretation can be done, and vice versa

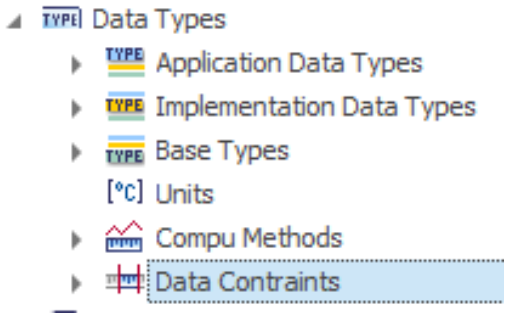
CW 26 2023 VI-S CEA Seitech

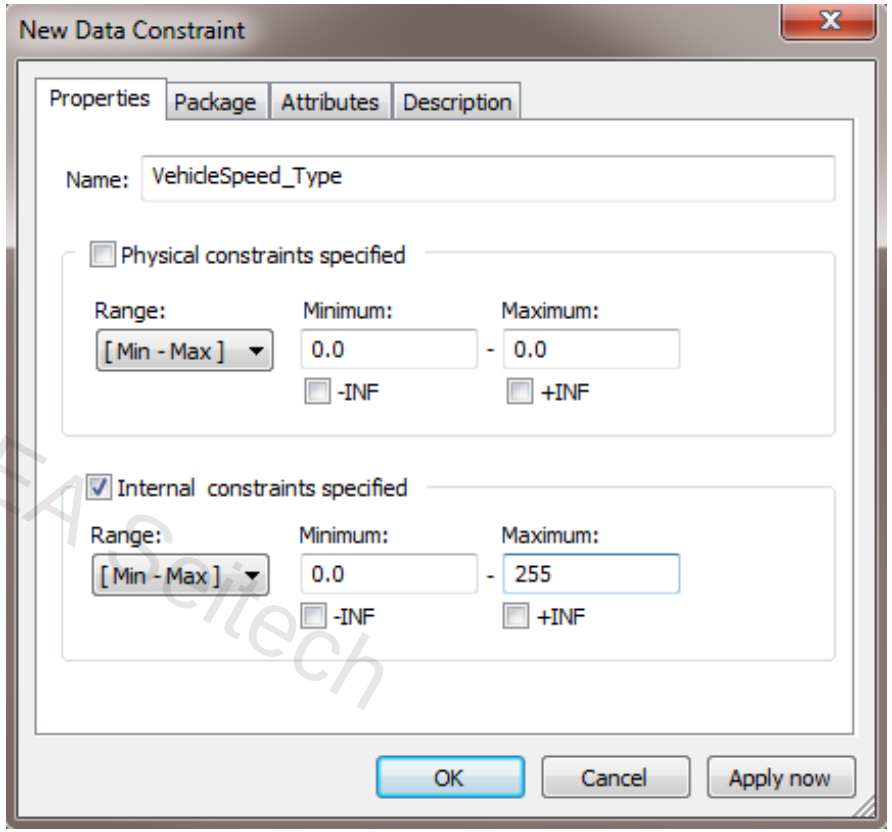
VECTOR

Data Constraints

- Data Constraints are globally defined
- Data Constraints apply to an Internal or Physical value
- ADTs and IDTs reference the data constraints

cw_26 2023 ; VI-S CEA Sciltech



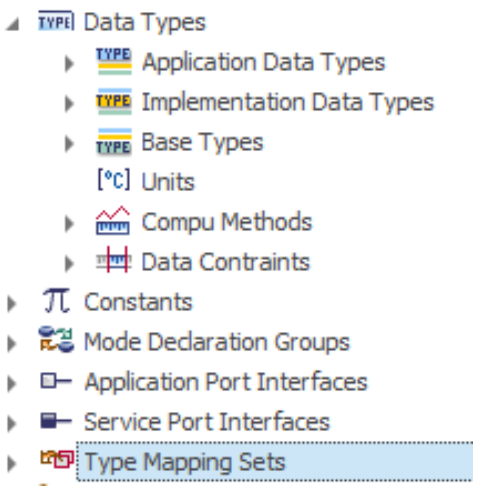


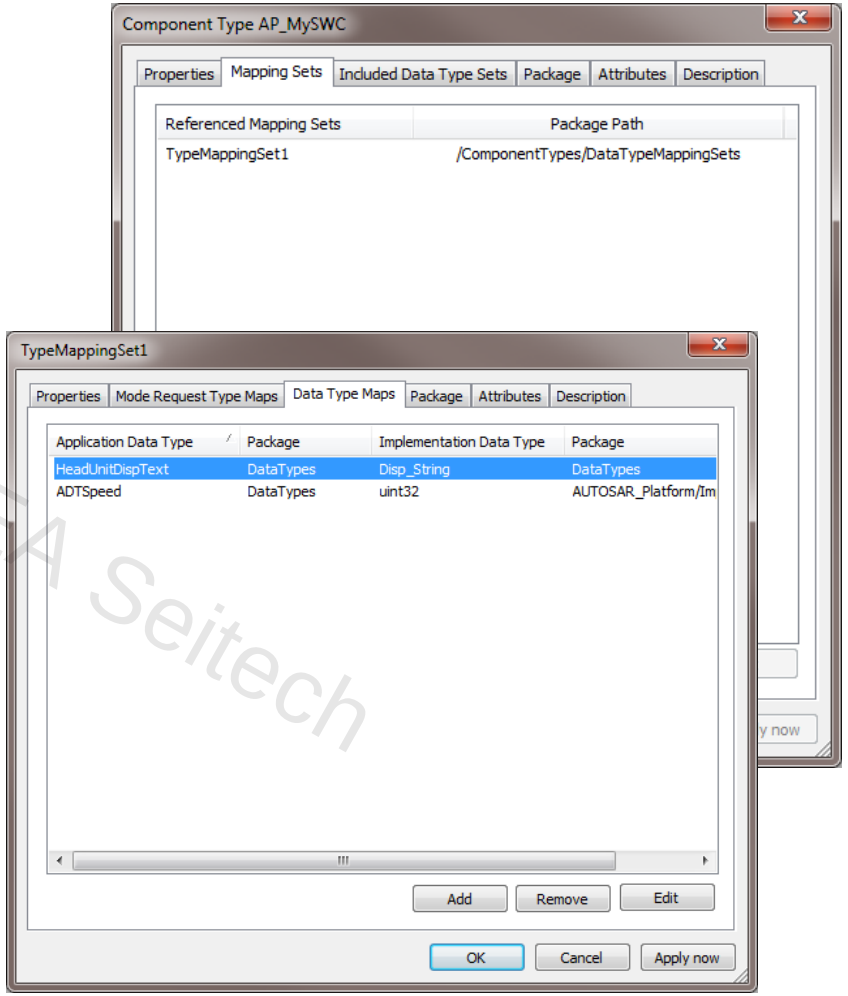
Defines Valid ranges for my Data Types

-> Replaces the invalid value from ASR3

Type Mapping Set

- Global definition of type mapping sets
  - Application Data Type → Implementation Data Type
  - Mode Declaration Group → Implementation Data Type
- Usage of type mapping set is in the context of an SWC
- Automatic Mapping function available (see next slide)



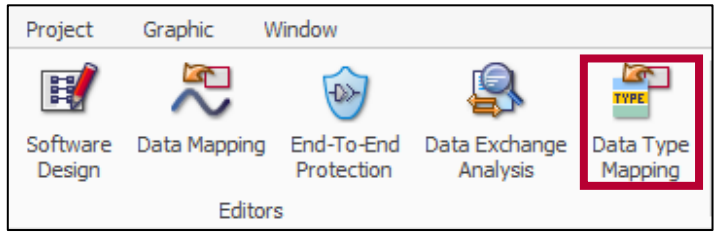


CW_26 2023 : VI-S CEA Seitech

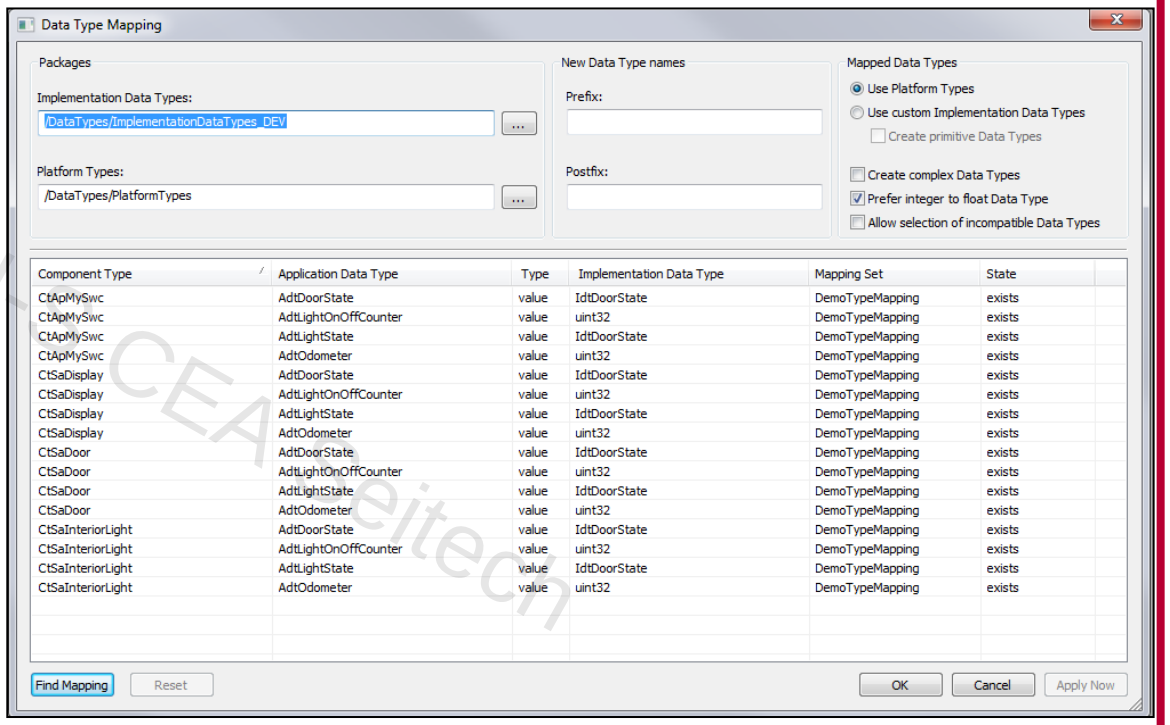
Type Mapping Set

The data type mappings can be derived automatically. Corresponding implementation data types will be selected or created.

ADT to IDT mapping is required for RTE generation.



Click inside the "Home" tab on "Data Type Mapping"



Component Type	Application Data Type	Type	Implementation Data Type	Mapping Set	State
CtApMySwc	AdtDoorState	value	IdtDoorState	DemoTypeMapping	exists
CtApMySwc	AdtLightOnOffCounter	value	uint32	DemoTypeMapping	exists
CtApMySwc	AdtLightState	value	IdtDoorState	DemoTypeMapping	exists
CtApMySwc	AdtOdometer	value	uint32	DemoTypeMapping	exists
CtSaDisplay	AdtDoorState	value	IdtDoorState	DemoTypeMapping	exists
CtSaDisplay	AdtLightOnOffCounter	value	uint32	DemoTypeMapping	exists
CtSaDisplay	AdtLightState	value	IdtDoorState	DemoTypeMapping	exists
CtSaDisplay	AdtOdometer	value	uint32	DemoTypeMapping	exists
CtSaDoor	AdtDoorState	value	IdtDoorState	DemoTypeMapping	exists
CtSaDoor	AdtLightOnOffCounter	value	uint32	DemoTypeMapping	exists
CtSaDoor	AdtLightState	value	IdtDoorState	DemoTypeMapping	exists
CtSaDoor	AdtOdometer	value	uint32	DemoTypeMapping	exists
CtSaInteriorLight	AdtDoorState	value	IdtDoorState	DemoTypeMapping	exists
CtSaInteriorLight	AdtLightOnOffCounter	value	uint32	DemoTypeMapping	exists
CtSaInteriorLight	AdtLightState	value	IdtDoorState	DemoTypeMapping	exists
CtSaInteriorLight	AdtOdometer	value	uint32	DemoTypeMapping	exists

The Data Type Mapping sets can automatically be created. This includes the resolution of which application data types map to which implementation data types. The implementation data types can also be created automatically.

cw 26 2023 ; VIN CEA Seitech

Example

ApplicationDataType

- Data constraint
  - [-50°C;+150°C]
  - In 0,5 °C steps
- Number of values required
  - 2 steps for 1 °C
  - 2*(150-(-50))+1= 401 values required

Problem 1:

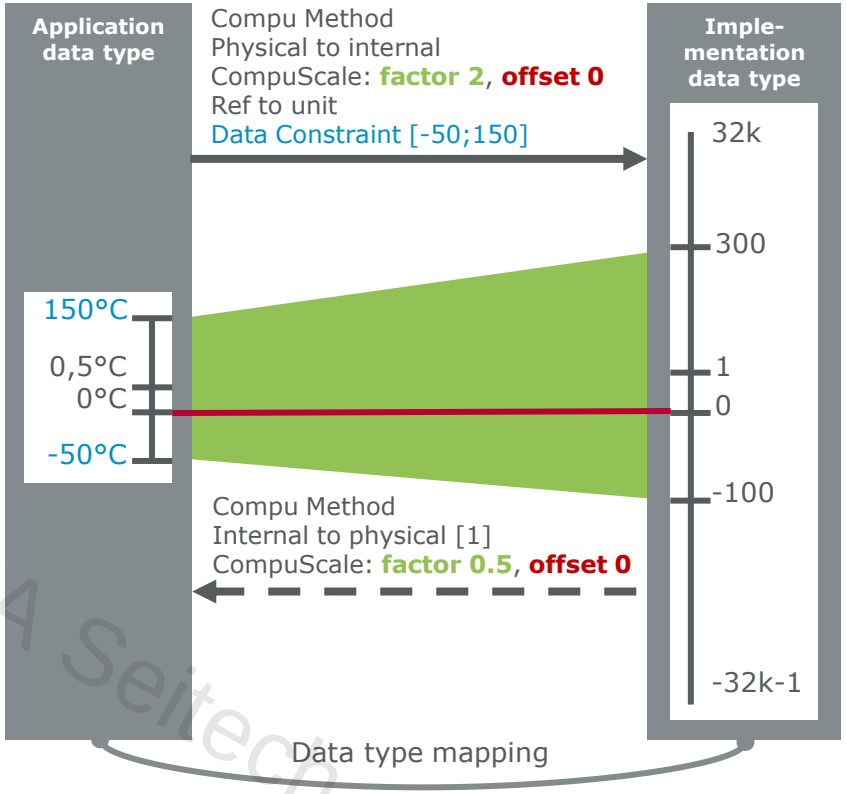
Make a choice for an **ImplementationDataType**

- uint8: 2^8 values = 256 values
  → too small
- uint16: 2^16 values = 65536 values → choice

Problem 2:

How could a 16 bit type represent the temperature range?

- Signed value range, 401 values
  - Take sint16
- But: Scaling rule required
  - Provided over the
    CompuMethod → Physical To Internal → CompuScale
  - Mapping to ImplementationDataType




```
ApplicationDataType

Name:                My_TemperatureType
Category:            Value, linear
CompuMethod:         Phys To Internal (…)
Data constraint:     [-50;150]
```

cw_26 2023 ; VI-S CEA Seitech

Example

[1] The physical to internal conversion is also used for internal to physical conversion in case the defined function is invertible. It is the case in this example, so it is enough to specify only the physical to internal conversion.

The linear conversion factor and offset are used in the following way: With Xi representing the internal value and Xp the physical one.

- Physical to internal: factor Fp and offset Op:

Xi = Fp.(Xp + Op)

- Internal to physical: factor Fi and offset Oi:

Xp = Fi.Xi + Oi

CW 26 2023 ; VI-S CEA Seitech

What becomes generated from which element?

Implementation Data Types

- Are the basis for the generation of “typedefs” on C-level (`Rte_Type.h`)
  - using existing platform types (`Platform_Types.h`)
  - defining own base types (`Rte_Type.h`)

Application Data Types

Are the basis for the generation of

- Init values
  - Calculated from
    - Init value in the computation method settings
- Lower and upper range limits
  - Preprocessor defines for access in the SWCs
  - Calculated from
    - constraint range, computation method
- Invalid value handling
- Enumerations
  - If defined computation method is a “text table”

SWZ2o 2023 ; VI-S CEA Seitech

Software Components

Exercise: SWCs

cw_26 2023, VI-S CEA Seitech